

Classical Planning in AI

Introduction to Classical Planning

Definition:

- Classical planning deals with finding a sequence of actions to achieve a goal in a fully observable, deterministic, finite, and static environment with *single-agent systems*. The key characteristics are:
 - **Fully Observable:** The agent knows the complete state of the world.
 - **Deterministic:** Each action has a single guaranteed outcome.
 - **Finite:** There are a fixed number of possible states.
 - **Static:** Only the agent's actions change the world.

Real-World Example: Package Delivery Robot

- **Scenario:**
- Robot needs to deliver packages between rooms
- Must plan sequence of movements and actions
- Consider:
 - Where robot is
 - Where packages are
 - Where packages need to go
 - Available paths between rooms

- A fundamental problem in AI: Finding a sequence of actions to achieve a goal.

Think of it like GPS navigation:

- Current location (Initial state)
- Destination (Goal state)
- Available roads (Actions)
- Finding the best route (Plan)

Core Components of a Planning Problem

- **Initial State:** Describes the starting conditions of the environment, e.g., “Plane1 is at AirportA, Cargo1 is loaded onto Plane1.”
- **Actions:** Actions transform one state to another, described using action schemas (general templates).
 - Example: *Fly(plane, from, to)*, where conditions (plane at “from” location) must be met.
- **Goal Test:** Specifies what the end goal looks like, e.g., “Cargo1 should be at AirportB.”
- **Result Function:** Defines the outcome of executing actions, including changes in state variables, e.g., flying the plane updates its location from AirportA to AirportB.

Representing States

- **Fluents:** Facts that can change
- **Propositional Logic:**
 - $\text{At}(\text{Robot}, \text{RoomA})$: Robot is in Room A
 - $\text{Have}(\text{Package})$: Robot has the package
 - $\text{Clean}(\text{RoomB})$: Room B is clean
- Initial State Example:
- $\text{At}(\text{Robot}, \text{RoomA}) \wedge \text{Have}(\text{Package}) \wedge \neg \text{Clean}(\text{RoomB})$

Representing Actions

Action Components

- 1. Name:** Identifier of the action
 - 2. Preconditions:** Required conditions
 - 3. Effects:** Changes to the state
 1. Add effects (positive)
 2. Delete effects (negative)
- Action: Move(robot, from, to)
 - PRECOND: $\text{At}(\text{robot}, \text{from}) \wedge \text{Path}(\text{from}, \text{to})$
 - EFFECT: $\neg \text{At}(\text{robot}, \text{from}) \wedge \text{At}(\text{robot}, \text{to})$

Importance:

- Planning is essential in AI for agents to act rationally to achieve goals, such as in robotics, automated navigation, or logistics.

Scope of Classical Planning:

- Works on *factored representations*, where the state of the world is represented as a collection of variables.
- Uses a language called *Planning Domain Definition Language (PDDL)* to describe initial states, goals, actions, and results in a structured way.

- **Factored Representations** in classical planning allow the state of the world to be represented by a collection of variables, rather than treating each state as an indivisible "atom" or singular unit. In a factored representation, each state is defined by the specific values of these variables, which enables more detailed and scalable problem-solving.

Key Points:

- **Factored Representation** involves breaking down the state into multiple attributes (variables or fluents), each with specific values that describe aspects of the world.
- This approach is more structured and scalable, as only relevant parts of the state change when actions are applied.

Example:

- Consider a **delivery robot** tasked with moving packages between two locations, RoomA and RoomB.
- **State Variables:**
 - Location(Robot): Specifies the robot's current location (RoomA or RoomB).
 - At(Package, Room): Indicates where the package is (RoomA or RoomB).
- **State Representation:**
 - A state might be represented as: $\text{Location(Robot)} = \text{RoomA} \wedge \text{At(Package, RoomB)}$.
 - This indicates the robot is in RoomA, while the package is in RoomB.
- **Action Example:**
 - Action(Move(Robot, from, to))
 - **Precondition:** $\text{Location(Robot)} = \text{from}$
 - **Effect:** $\text{Location(Robot)} = \text{to}$
 - If the robot moves from RoomA to RoomB, only Location(Robot) changes, making the transition efficient in terms of only updating the affected variables.
- Using factored representation, actions only alter specific state variables rather than redefining the entire state, simplifying and accelerating the planning process.

Representation in PDDL (Planning Domain Definition Language)

- **PDDL Basics:** Represents states as *conjunctions of fluents* (facts that are true in a state).
- Fluents use a closed-world assumption, meaning any unspecified fluent is false.
- **Example:** Action(Fly(p, from, to),
 - Precondition: $\text{At}(p, \text{from}) \wedge \text{Plane}(p) \wedge \text{Airport}(\text{from}) \wedge \text{Airport}(\text{to})$
 - Effect: $\neg \text{At}(p, \text{from}) \wedge \text{At}(p, \text{to})$
- **Use Case:** This schema allows defining actions concisely without listing every possible combination.

Example Problem – Air Cargo Transport

- **Scenario:** Objective: Move cargo between airports using available planes.
- **Initial State:** e.g., Plane1 and Cargo1 are at AirportA, Cargo2 at AirportB.
- **Goal State:** Cargo1 at AirportB and Cargo2 at AirportA.
- **Actions:** *Load*, *Unload*, and *Fly* actions move cargo from one location to another.
- **Solution Example:** [Load(Cargo1, Plane1, AirportA), Fly(Plane1, AirportA, AirportB), Unload(Cargo1, AirportB)]

Action Schemas and Frame Problem

- **Action Schemas:** Templates define actions, reducing repetitive code.
- Example: *Fly(p, from, to)* schema covers any plane moving between any two airports.
- **Frame Problem:** Describes challenges in specifying only the changes caused by actions.
- PDDL resolves this by focusing on the effects of actions (what changes) and assumes all else remains unchanged.

Example – Spare Tire Problem

- **Problem Setup:** Goal: Replace a flat tire with a spare.
- **Initial State:** Flat tire on axle; spare in trunk.
- **Goal State:** Spare tire on axle.
- **Actions:** *Remove, PutOn, LeaveOvernight*
- The effect of leaving the car overnight causes the tires to disappear, adding a constraint.
- **Solution Sequence:**[Remove(Flat, Axle), Remove(Spare, Trunk), PutOn(Spare, Axle)]

Block World Problem

- **Problem Description:** Arrange blocks in a specific order on a table using a robotic arm.
- **Initial and Goal States:** Initial: Block C on A, Block A on table.
- Goal: Stack blocks in the order A on B on C.
- **Actions:** *Move(block, from, to)* where constraints specify only one block can be moved at a time.
- **Solution Example:**[Move(C, A, Table), Move(A, Table, B), Move(B, Table, C)]

Algorithms for Planning as State-Space Search

- **Two Types of Search:** ***Forward Search:*** Starts at the initial state and moves toward the goal.
- ***Backward Search:*** Begins from the goal and works back to the initial state.
- **Comparison:** Forward is intuitive but can lead to many irrelevant actions.
- Backward is efficient for goal-focused actions but can be harder to set up.
- **Example:** Forward search explores all possible steps from starting configuration until reaching the goal.

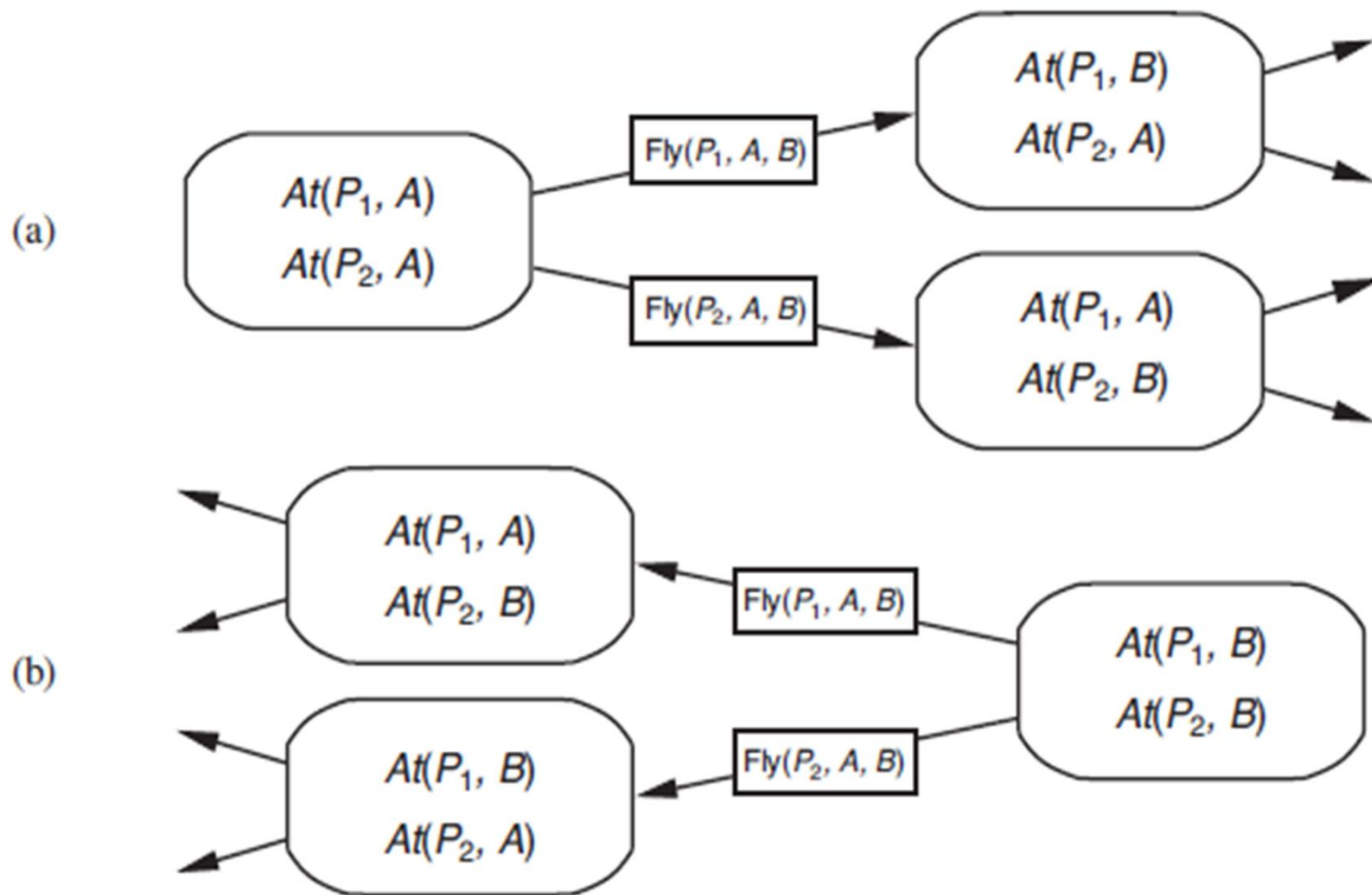


Figure 10.5 Two approaches to searching for a plan. (a) Forward (progression) search through the space of states, starting in the initial state and using the problem's actions to search forward for a member of the set of goal states. (b) Backward (regression) search through sets of relevant states, starting at the set of states representing the goal and using the inverse of the actions to search backward for the initial state.

Heuristics in Planning

- **Role of Heuristics:** Estimate distance from current state to goal, guiding search efficiency.
- **Common Heuristics:** *Ignore Preconditions:* Assumes every action is always possible, giving a minimal action count.
- ***Ignore Delete Lists:*** Actions don't negate each other, simplifying the action sequence.
- **Application Example:** In the air cargo problem, ignoring delete lists allows loading/unloading without constraint checking, reducing complexity.

Planning Graphs

- **Definition:** A planning graph is a layered structure representing possible states and actions over time.
- **Levels:** Alternate between *state levels* (states that can be achieved) and *action levels* (actions that can be taken).
- **Example:** In a delivery problem, the planning graph shows each state transition as layers, helping visualize and solve complex plans.

Planning Graph Example – Have Cake and Eat It Too

- **Problem Setup:**Goal: Have a cake and eat it too.
- **Initial State:** Cake exists but isn't eaten.
- **Actions:** *Eat* or *Bake* to either consume or replenish the cake.
- **Graph Analysis:**Shows levels where either action can occur, with mutual exclusions between states.
- Visualization helps resolve logical conflicts like eating a cake without baking a new one.

Summary and Comparison of Classical Planning Approaches

- **Overview of Techniques:** *State-space search*: Both forward and backward.
- *Planning graphs*: Efficient, layered representation for complex problems.
- *Satisfiability (SAT) approaches*: Translate problems to Boolean satisfiability.
- **Use Cases and Examples:** *State-space search* is practical in small, well-defined problems, while *planning graphs* are effective in complex domains.

Applications of Classical Planning in AI

- **Practical Applications:** *Robotics:* Autonomous task planning, such as factory robots.
- *Logistics:* Package delivery, route optimization.
- *Games and Simulations:* AI agents in games follow planning techniques to achieve goals.
- **Example:** Using classical planning, a robot can be programmed to clean an area by efficiently navigating and picking up obstacles.

Introduction to Planning Graphs

- A **planning graph** is a data structure used in **artificial intelligence (AI) planning** to represent possible actions and states over time.
- It is particularly effective in **classical planning** problems, where an agent seeks a sequence of actions that will transition from an initial state to a goal state.
- Planning graphs were introduced by **Avrim Blum and Merrick Furst** in 1995 in their **Graphplan** algorithm, which is still influential in automated planning.

Key Concepts of a Planning Graph

- **Levels:** A planning graph is structured into alternating levels of **state (or literal) layers** and **action layers**.
 - **Literal layers** represent all possible facts (or propositions) that could be true at that step.
 - **Action layers** represent all possible actions that could be applied at that step.
- **Expansion:** The graph is constructed layer by layer, starting from the initial state:
 - Each literal layer is followed by an action layer, representing all actions that can occur based on the literals in the previous layer.
 - Each action layer is followed by a new literal layer, representing the possible effects of actions in the previous layer.

- **Mutex (Mutual Exclusion) Relations:**
 - Planning graphs track **mutual exclusions** or **mutexes**, which are pairs of actions or literals that cannot coexist.
 - Mutexes help limit the planning graph's search space by ruling out incompatible actions or states, simplifying goal achievement checks.
- **Termination:**
 - The planning graph stops growing when a **fixed point** is reached (no new literals or actions can be added).
 - At this point, if the goal is not achieved, it is often impossible to achieve it with the given actions.

Structure of a Planning Graph

- **Level 0 (Initial State):** Contains the literals of the initial state.
- **Level 1 (First Action Level):** Contains actions that can be applied to the literals in the initial state.
- **Level 2 (First Literal Level):** Contains the literals resulting from the effects of actions in Level 1.
- **Subsequent Levels:** Alternate between action and literal layers, incorporating mutexes at each level.

Uses of Planning Graphs

- **Goal Testing:** Planning graphs allow quick checks to determine if the goal is achievable in a given level.
- **Heuristic Search:** Planning graphs can be used to derive heuristics for **planning algorithms** like **A*** by estimating the cost of reaching the goal.
- **Parallel Planning:** Planning graphs help find action sequences by considering actions that can be performed in parallel, as long as they're not mutually exclusive.

Graphplan Algorithm

- The **Graphplan** algorithm utilizes the planning graph structure to:
 - Build a planning graph up to a fixed point.
 - Use a **backward search** to check for a sequence of non-mutex actions that lead to the goal state.
- This method enables Graphplan to efficiently find a solution or determine if one does not exist, even in complex planning domains.

Applications

- Planning graphs are used in:
 - **Automated Planning:** For example, in robotics and AI agents that need to plan a sequence of actions.
 - **Heuristic Development:** Planning graphs provide informative heuristics in state-space search.
 - **Multi-agent Systems:** To manage parallel actions and constraints across agents.
- Planning graphs are a foundational concept in classical AI planning, providing both a powerful structure for representing possible actions and a practical tool for deriving efficient heuristics in automated planning.

The **Graphplan** algorithm

- The **Graphplan** algorithm is a **planning algorithm** that uses a data structure called a **planning graph** to find a solution to classical planning problems.
- Developed by **Avrim Blum and Merrick Furst** in 1995, it introduced a new way of solving planning problems more efficiently by alternating between **planning graph expansion** and **backward search** for a solution.

Key Components of Graphplan

- **Planning Graph:** A layered graph that alternates between levels of **actions** and **states (literals)**.
 - **Action levels** represent all actions that could be taken given the current set of possible states.
 - **Literal levels** represent all possible states resulting from the previous level's actions.
- **Mutex (Mutual Exclusion) Relations:** These are constraints that prevent certain actions or states from occurring together. Graphplan uses mutexes to reduce the search space by identifying incompatible actions or states early in the process.
- **Fixed Point:** The planning graph grows until a **fixed point** is reached, meaning no new actions or literals can be added. If the goal has not been achieved by this point, then no solution exists.

Graphplan Algorithm Steps

- The Graphplan algorithm has two main phases:

1. Graph Expansion(Construction) Phase

2. Solution Extraction Phase

Step 1: Graph Expansion Phase

- **Initialize the Planning Graph:** Start with an initial state as **Level 0** of the planning graph, which is a literal level containing the propositions (literals) that are true in the initial state.
- **Expand Levels:**
 - **Alternate Levels:** Add an **action level** followed by a **literal level**.
 - **Action Level:** Add all actions that can be performed based on the literals in the current literal level. Actions can only be included if their preconditions are met by literals in the previous level.
 - **Literal Level:** Add all literals that are the effects of actions from the previous action level.

- **Add Mutex Relations:**
 - Between **actions** in an action level if they interfere with each other. For example, if one action deletes a precondition required by another action.
 - Between **literals** in a literal level if they cannot be achieved simultaneously.
- **Repeat Expansion:**
- Continue expanding the graph by adding alternating action and literal levels until either:
 - The literals in the literal level include all the literals of the goal (meaning a solution might exist), or
 - A fixed point is reached (no new literals or actions are added between levels), indicating that further expansion will not provide any new information.

Example - The “have cake and eat cake too” problem.

Init(Have(Cake))

Goal(Have(Cake) $\wedge$ Eaten(Cake))

Action(Eat(Cake))

PRECOND: *Have(Cake)*

EFFECT: $\neg$ *Have(Cake) $\wedge$ Eaten(Cake)*

Action(Bake(Cake))

PRECOND: $\neg$ *Have(Cake)*

EFFECT: *Have(Cake)*

S_0

A_0

S_1

Have(Cake)

$\neg$ *Eaten(Cake)*

S_0

A_0

S_1

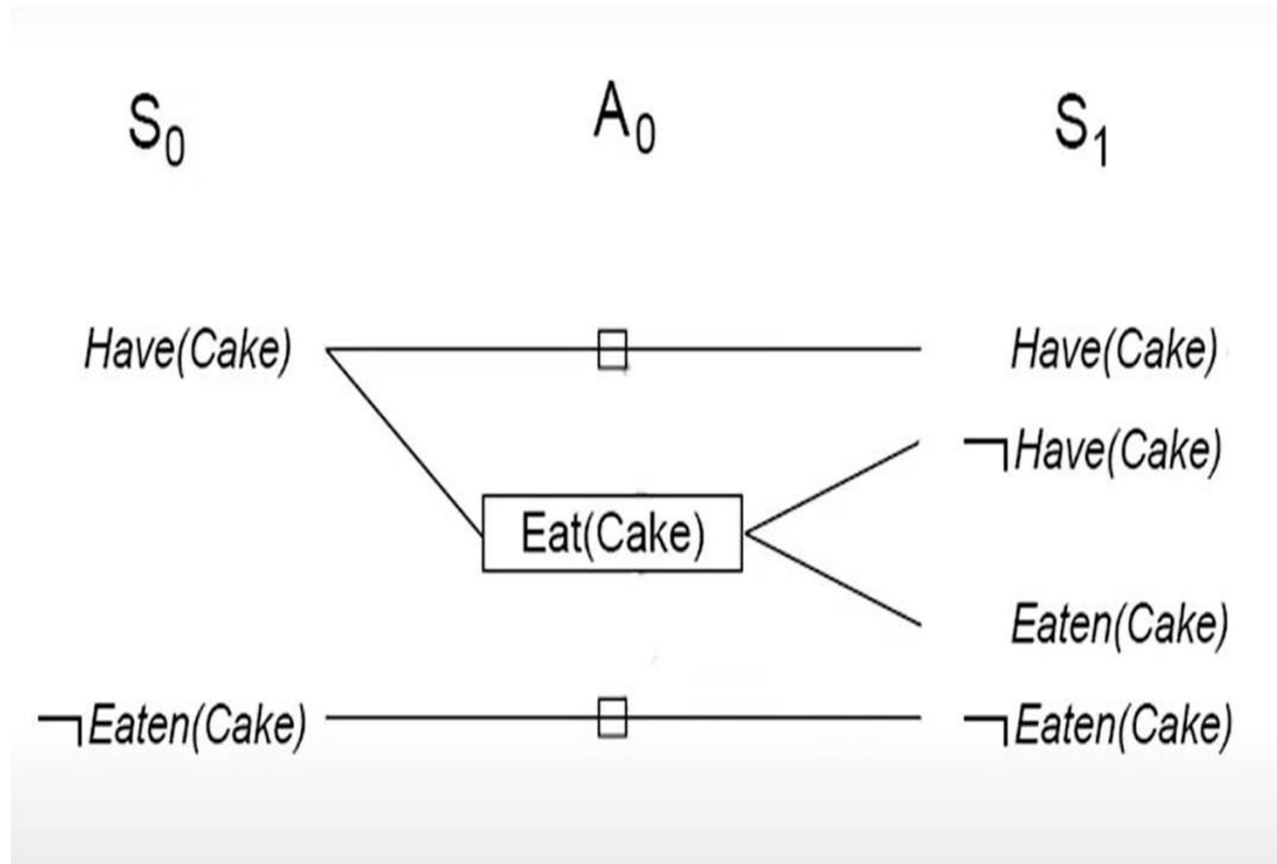
$Have(Cake)$

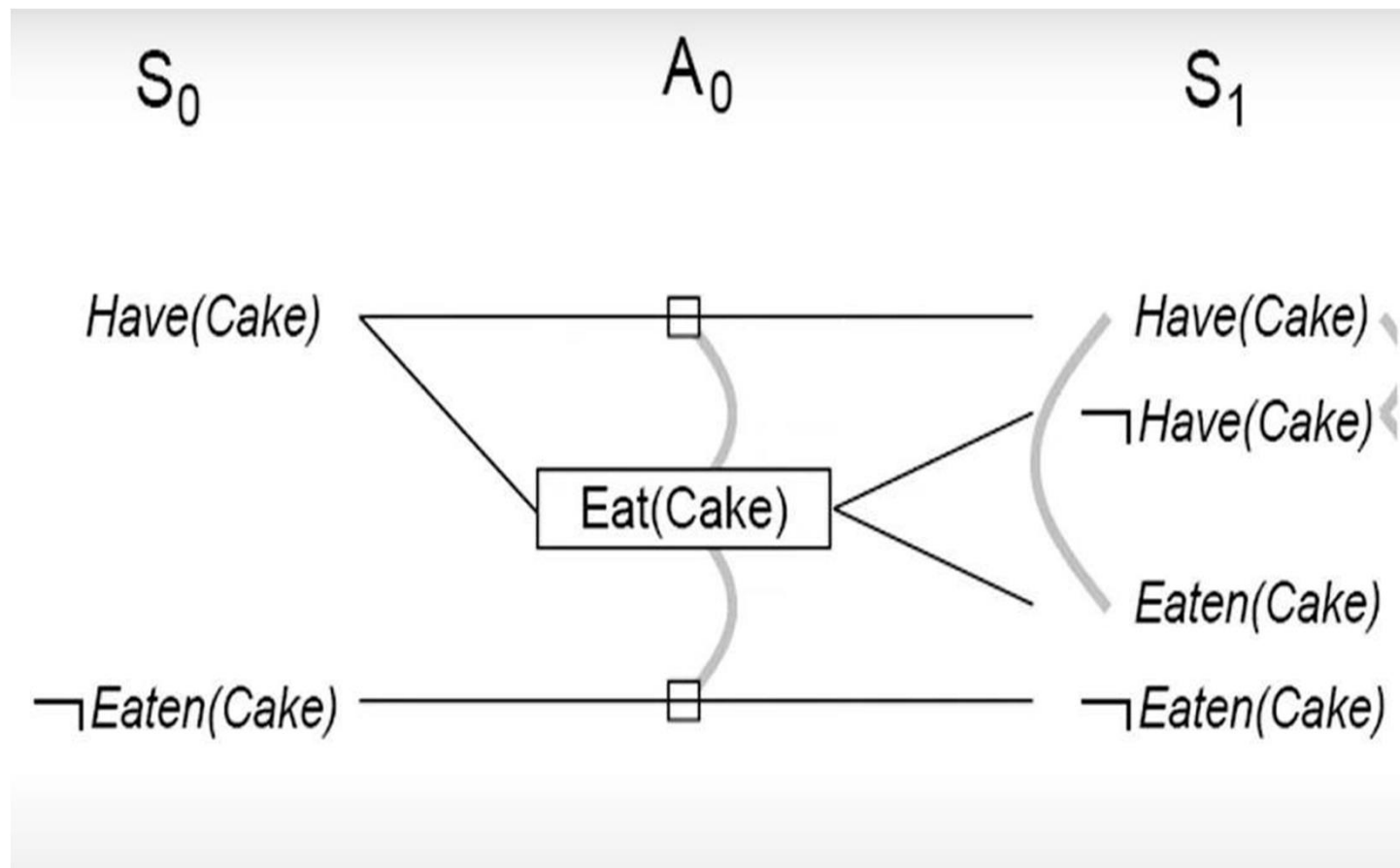
$Eat(Cake)$

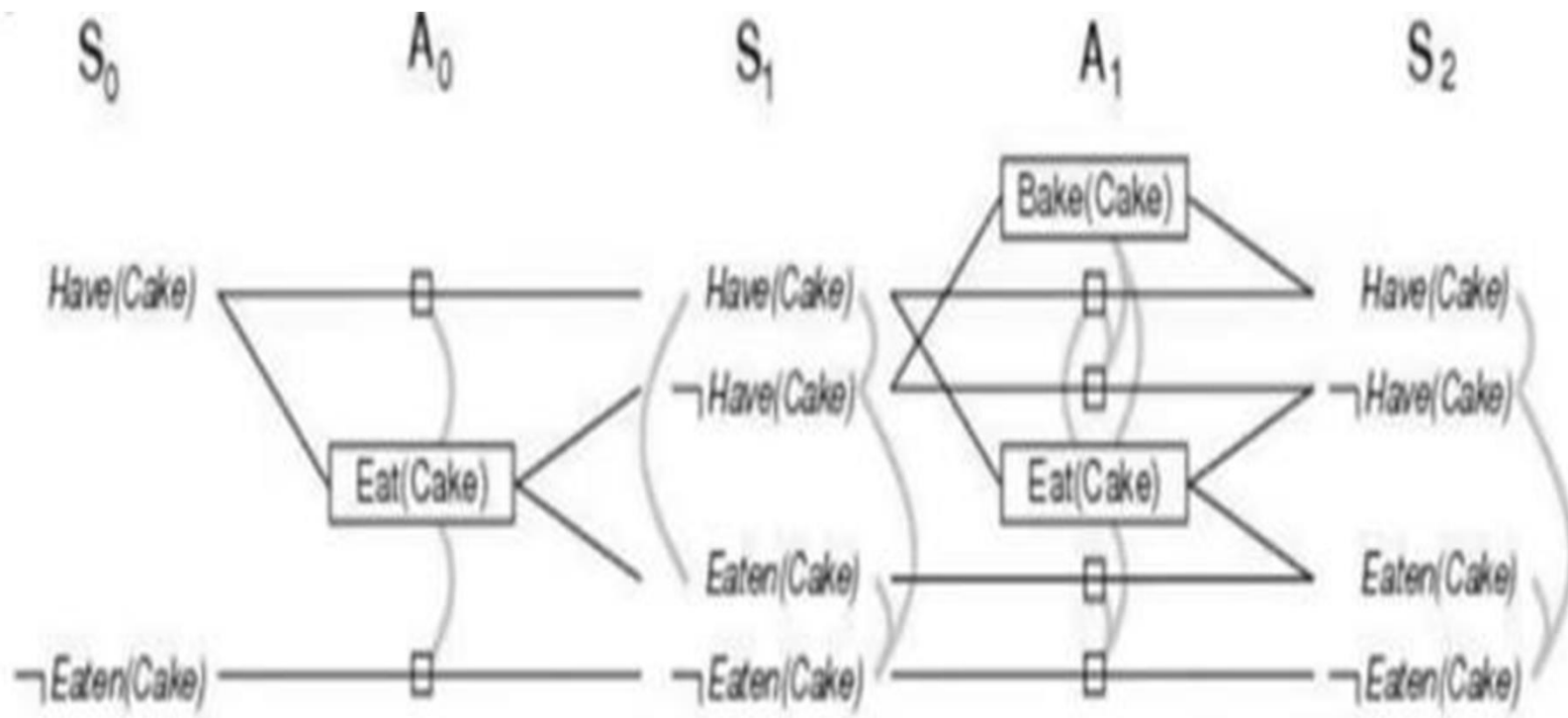
$\neg Have(Cake)$

$Eaten(Cake)$

$\neg Eaten(Cake)$







Step 2: Solution Extraction Phase (Backward Search)

Once the planning graph is expanded, Graphplan attempts to extract a solution from it using **backward search**.

1. Check Goal Achievement:

1. Starting from the last literal level, check if the literals in this level include the **goal literals** (the conditions that define the goal state).
2. If all goal literals are present and not mutex with each other, the graph may contain a solution.

2. Backtracking Search:

1. Begin at the last literal level and **work backward** to find a set of non-mutex actions that can achieve the goal literals at each level.
2. For each literal in the goal, select actions from the previous action level that produce the required literals as effects.
3. Each selected action must not be in a mutex relationship with any other selected action.

Recursive Backtracking:

- Repeat the process for each action's preconditions by moving backward through the levels until reaching Level 0.
- If this backward search is successful without mutex conflicts, then the series of actions found is a valid **solution plan**.

Failure of Solution Extraction:

- If a valid set of actions cannot be found due to mutex conflicts or missing literals, the algorithm returns to the **Graph Expansion Phase** and expands the planning graph by one additional level, then reattempts the Solution Extraction Phase.

Termination:

- If the algorithm reaches a fixed point during graph expansion and no solution is found, it concludes that the goal is not achievable, and **no solution exists** for the problem.

The GRAPHPLAN Algorithm

function GRAPHPLAN(*problem*) **returns** solution or failure

graph $\leftarrow$ INITIAL-PLANNING-GRAPH(*problem*)

goals $\leftarrow$ GOALS[*problem*]

loop do

if *goals* all non-mutex in last level of *graph* **then do**

solution $\leftarrow$ EXTRACT-SOLUTION(*graph*, *goals*, LENGTH(*graph*))

if *solution* $\neq$ failure **then return** *solution*

else if NO-SOLUTION-POSSIBLE(*graph*) **then return** failure

graph $\leftarrow$ EXPAND-GRAPH(*graph*, *problem*)

Figure 11.13 The GRAPHPLAN algorithm. GRAPHPLAN alternates between a solution extraction step and a graph expansion step. EXTRACT-SOLUTION looks for whether a plan can be found, starting at the end and searching backwards. EXPAND-GRAPH adds the actions for the current level and the state literals for the next level.

Introduction to Hierarchical Planning

- **What is Hierarchical Planning?** Hierarchical planning is a technique in AI where a complex problem is broken down into simpler subtasks at multiple levels of abstraction.
- **Purpose:** To manage and solve large problems efficiently by working through smaller, organized components.
- **Example:** In a **robotics application**, the high-level task “Clean the room” can be broken down into “Vacuum the floor,” “Dust surfaces,” and “Organize items,” which are then further divided into smaller, manageable steps.

Why Use Hierarchical Planning?

- **Simplifies Complex Problems:** Reduces the planning complexity by organizing tasks into hierarchies, making them easier to understand and solve.
- **Improves Efficiency:** AI can address smaller tasks independently, making the overall solution faster.
- **Example:** In **autonomous driving**, the high-level task “Drive to destination” can be decomposed into “Navigate streets,” “Follow traffic rules,” and “Park safely.”

How Hierarchical Planning Works

- **Step 1: Define High-Level Goals** Identify the overall task (e.g., “Plan a wedding”).
- **Step 2: Decompose into Subtasks** Break the task into main components: “Book venue,” “Arrange catering,” “Send invitations.”
- **Step 3: Break Subtasks Further** For example, “Book venue” can be further divided into “Choose location,” “Contact venue,” and “Finalize booking.”
- **Example Hierarchy:**
- High-Level Goal: **Plan a wedding**
 - Subtasks: Book venue, Arrange catering, etc.
 - Sub-subtasks: Choose location, Contact venue, etc.

Key Components of Hierarchical Planning

- **Hierarchy of Abstractions:** Tasks are structured at different levels of detail.
- **Task Decomposition:** Each high-level task is broken into subtasks until they're simple enough to execute.
- **Example: Cooking a meal:**
 - High-level task: "Cook a three-course dinner"
 - Decomposed tasks: "Prepare appetizer," "Cook main course," "Make dessert"
 - Further breakdown: "Chop vegetables," "Boil pasta," etc.

Types of Hierarchical Planning Approaches

- **Hierarchical Task Networks (HTN):** Uses a network structure where each node represents a task or subtask.
- **Example:** In HTN planning for a rescue robot, tasks like “Find victim” can be broken down into steps like “Scan area,” “Approach location,” and “Assist victim.”
- **Abstraction-Based Planning:** Simplifies tasks by grouping similar actions together at higher levels.

Hierarchical Task Networks (HTN) in Detail

- **Definition:** HTN planning is an approach where tasks are represented as networks, decomposing tasks recursively.
- **Process:** Define high-level tasks and break them down into ordered sequences of actions.
- **Example:**
- High-level task: “Prepare for an exam”
 - Tasks: “Review notes,” “Take practice tests,” “Rest before exam”
 - Subtasks for “Review notes”: “Go over chapters,” “Make summaries,” etc.

Benefits of Hierarchical Planning

- **Scalability:** Can handle complex and large-scale tasks efficiently.
- **Reusability:** Subtasks can often be reused in different contexts.
- **Adaptability:** Easily modify parts of the hierarchy without affecting the whole structure.
- **Example:** In **manufacturing**, “Build a car” can be broken into reusable tasks like “Assemble chassis,” “Install engine,” and “Paint exterior,” which are consistent across various models.

Challenges of Hierarchical Planning

- **Task Dependency Issues:** Some tasks may depend on others, complicating scheduling.
- **Complexity in Large Hierarchies:** Deep hierarchies can be difficult to manage.
- **Example:** For an **AI system managing inventory**, if “Restock item” depends on “Check stock levels,” delays in checking stock can hold up the entire process.

Applications of Hierarchical Planning

- **Robotics:** Robots performing tasks like cleaning, object manipulation, or navigation.
- **Game AI:** Characters in video games use hierarchical planning to make complex decisions.
- **Example:** In a **video game**, the AI for a character can plan high-level actions like “Patrol area” and decompose it into “Walk route,” “Check for enemies,” and “Return to base.”

Real-World Example: Autonomous Driving

- **High-Level Goal:** “Drive from point A to point B” Subtasks: “Start car,” “Navigate route,” “Follow traffic rules,” “Park”
- **Subtask Breakdown:** “Navigate route” further breaks down into “Turn at intersections,” “Avoid obstacles,” and “Merge into lanes.”
- **Benefit:** By planning hierarchically, the car can focus on immediate steps within a larger goal.